

CST8507: Natural Language Processing

Lab 4: Sentiment Analysis using DistilBERT

Objective

This lab will provide you with experience with the following:

- How to prepare text data for sentiment analysis with deep learning methods.
- How to develop a neural Sentimental model with word embedding and Deep NN
- How to evaluate the developed a sentimental model.
- How to develop a neural Sentimental model with Transformers

Description

Important note : Breaking down the code into manageable sections based on the assignment requirements and incorporating checkpoints to verify correctness is crucial. This approach not only helps you stay organized but also significantly impacts your grades for this assignment.

Dataset

Download Movie Review Polarity Dataset (review_polarity.tar.gz, 3MB).

http://www.cs.cornell.edu/people/pabo/movie-review-data/review_polarity.tar.gz

Once you extract the files from the zipped folder, you will notice a new directory named "txtsentoken" which comprises two subdirectories named "neg" and "pos" for negative and positive reviews respectively. Each review is saved as an individual file with a specific name format "cv000" to "cv999" for both negative and positive reviews.

Text Preprocessing

Preparing text data for feature engineering and modeling by cleaning your data. Examine your data and determine which preprocessing steps are necessary to clean your text data.

Define a Vocabulary

Defining a vocabulary of known words is a crucial step in text modeling as it can impact the accuracy and efficiency of the model.

- To create the vocabulary, we can iterate through all the reviews in the negative directory followed by the positive directory and add each document to the vocabulary Counter.

After running the previously mentioned step, we can observe that the resulting vocabulary contains approximately **44,276** unique words. This vocabulary was constructed by iterating through only the reviews in the training dataset and adding each document to a Counter.

- Print a sample of the top 50 most frequently occurring words in the reviews can be printed for further analysis. It is important to note that this vocabulary is specific to the training dataset and may not represent all possible words that could be encountered in the movie reviews.

```
44276
[('film', 7983), ('one', 4946), ('movie', 4826), ('like', 3201), ('even', 2262), ('good',
 2080), ('time', 2041), ('story', 1907), ('films', 1873), ('would', 1844), ('much',
 1824), ('also', 1757), ('characters', 1735), ('get', 1724), ('character', 1703),
 ('two', 1643), ('first', 1588), ('see', 1557), ('way', 1515), ('well', 1511), ('make',
 1418), ('really', 1407), ('little', 1351), ('life', 1334), ('plot', 1288), ('people',
 1269), ('could', 1248), ('bad', 1248), ('scene', 1241), ('movies', 1238), ('never',
 1201), ('best', 1179), ('new', 1140), ('scenes', 1135), ('man', 1131), ('many', 1130),
 ('doesnt', 1118), ('know', 1092), ('dont', 1086), ('hes', 1024), ('great', 1014),
 ('another', 992), ('action', 985), ('love', 977), ('us', 967), ('go', 952),
 ('director', 948), ('end', 946), ('something', 945), ('still', 936)]
```

Pruning

- To improve the quality and effectiveness of the vocabulary for text modeling, we can perform a pruning step to remove words with low occurrences. One approach is to iterate through the vocabulary and remove all words that have a low frequency, such as those that appear only once or twice in all the reviews. By pruning the vocabulary, we can reduce the number of irrelevant and noisy words, resulting in a more concise and effective set of words for use in text modeling.
- After completing the pruning step to remove low-frequency words, the resulting vocabulary can be saved to a new file named "vocab.txt". This file can be later loaded and used to filter movie reviews before encoding them for modeling purposes.

Upon executing the above steps with the addition of saving the vocabulary to a new file, we can observe a significant reduction in the vocabulary size. Specifically, the size decreases from 44,276 words to **25,767** words, which is a reduction of more than half its original size. This reduction in vocabulary size can lead to faster and more efficient text modeling while still maintaining a high level of accuracy.

Baseline Model: TF-IDF +logistic Regression

Before fine-tuning DistilBERT, you are required to implement and evaluate a classical baseline model (e.g., TF-IDF + Logistic Regression). This step is essential for scientific comparison. A transformer model must demonstrate meaningful improvement over simpler approaches to justify its computational cost and complexity. By comparing DistilBERT against a baseline model, you will assess whether contextual language modeling improve the performance of sentimental analysis task, or whether lexical features alone are sufficient. Split the data into 80% training and 20% testing sets.

Report the following measures:

1. Accuracy
2. Precision
3. Recall
4. F1-score
5. Confusion matrix

Fine-Tuning DistilBERT

The main advantage of this model is that it achieves comparable performance to BERT, while being significantly smaller and more efficient. This enables us to train a classifier in a few minutes.

Steps:

1. Install Required Libraries: Ensure you have the necessary libraries installed.
2. Load and Preprocess the Dataset.
3. Split the Dataset: Divide the dataset into training, validation, and test sets.
4. Load DistilBERT Model:
5. Use the DistilBERT model from the Transformers library. You can use the `DistilBertForSequenceClassification` class.
6. Tokenize and Encode Text: Tokenize and encode the text data using the DistilBERT tokenizer.
7. Fine-tune the Model: Fine-tune the DistilBERT model on the training dataset. Use a suitable optimizer and loss function.

8. Evaluate the Model: Evaluate the model on the validation set and fine-tune as needed.

You must report:

1. Accuracy
2. Precision
3. Recall
4. F1-score
5. Confusion matrix

9. Use the trained model for making predictions on new text samples,

```
new_text_samples = [  
    "This book was great!",  
    "This book was terrible!",  
    "To say that this is the worst story ever is to insult the worst  
story ever!",  
    "I loved this book!",  
    "It was so-so!",  
    "This novel was good enough for me!",  
    "Total piece of garbage",  
    "Most amazing stories ever that inspired me to great things!",  
    "It would be neither confusing nor coherent to describe the nove  
l in some optimal way that does not distract from its idiocratic syn  
chronicity"  
]
```

Code design and style Requirements

Make sure that your program is properly documented:

- You should have a docstring at the very beginning of the file briefly describing your program and stating your name, section and creativity additions.
- Each function should have an appropriate docstring (including arguments and return value if applicable).
- Other miscellaneous comments to make things clear

In addition, make sure that you've used good code design and style (including meaningful variable names, constants where relevant, vertical white space, etc.).

Submit your code and report via Brightspace. Your program file must be named "**lab4_Class**" You can submit multiple times, with only the most recent submission (before the due date) graded.

Submission Instruction

Instructions for Adding Headers in Jupyter Notebook Code

For better organization and readability of your code, please include **clear headers** at the beginning of each task or section in your Jupyter notebook. These headers should:

1. **Describe the Task:** Clearly state the task or step the code is performing.
2. **Use Markdown Cells:** Use Markdown cells (not code cells) to create headers, so they stand out visually.
3. **Format the Header:** Use appropriate Markdown syntax for headers (e.g., # for main titles, ## for subheadings).

Submit your code as a Jupyter Notebook with the running code, you can do the following steps:

1. **Open your Jupyter Notebook:** You can open Jupyter Notebook either from the command line by typing "jupyter notebook" or from Anaconda Navigator.

2. Write your code: Write the code you want to submit in the cells of the Jupyter Notebook. **Make sure to run the cells so that the output is generated.**
3. Save the Notebook: Once you have written and run your code, save the Jupyter Notebook by clicking on File -> Save and Checkpoint or by pressing "Ctrl + S".
4. Export the Notebook: To export the Jupyter Notebook, go to File -> Download as -> Notebook (.ipynb). This will download the Notebook as a .ipynb file on your laptop.
5. Create a folder named "Lab4", Inside the " Lab4" folder, include:

- o "Lab4_Code.ipynb "

submit your file via Brightspace. You can submit multiple times, with only the most recent submission (before the due date) graded. There will be mark deduction if you are not following the submission requirements.

Grading Criteria

Criteria	Points
Text Preprocessing and Pruning	10
Implementing Baseline Model	20
Implementing Fine-Tuning DistilBERT	60
Evaluation, testing and reporting results	10

Due Date

Check Brightspace for due dates.